Software Requirements Specification

for

Centralized Mess Management System (CMMS)

Version 1.0

Prepared by

Group #: 19 **Group Name: FullThrottle Tenacious Dynasty**

Aaditya Bijarniya	230004	aadityab23@iitk.ac.in
Shubham Kumar Patel	241010	shubhamkp24@iitk.ac.in
Dev Prakash	240338	devp24@iitk.ac.in
Shital Niras	230967	shitalan23@iitk.ac.in
Kavita Kumari	230552	kavitak23@iitk.ac.in
Soumyaranjan Sahu	241035	soumyasahu24@iitk.ac.in
Deepak Kumar	240329	deepakk24@iitk.ac.in
Ashwina Yadav	230235	ashwinay23@iitk.ac.in
Itesh Mishra	230489	iteshm23@iitk.ac.in
Tanush Khagwal	231083	tanushk23@iitk.ac.in

Course: CS253

Mentor TA: Vinayak Bhosle

Date: 24 January 2026

CONTENTS

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Aaditya Bijarniya Shubham Kumar Dev Prakash Shital Anurath Niras Kavita Kumari Tanush Khagwal Deepak Kumar Soumyaranjan Sahu	First Draft of SRS for the project CMMS.	24/01/26

Introduction

1.1 Product Scope

The **Centralized Mess Management System (CMMS)** is a comprehensive web-based application designed to digitize and unify dining operations across the various halls of residence at IIT Kanpur. Currently, mess management relies on disjointed manual processes such as physical registers for "extras," paper-based rebate forms, and decentralized menu announcements resulting in long queues, accounting discrepancies, and significant food wastage.

The primary objective of CMMS is to streamline these operations through two distinct interfaces: a **Student Portal** and a **Manager Portal**. The system allows students to view centralized menus, pre-book extra items via QR code generation, and automate rebate applications. Simultaneously, it empowers mess managers to validate orders via QR scanning, manage live inventory to prevent overbooking, and utilize analytics to forecast food requirements. The key benefits of CMMS include the elimination of physical queues, accurate expense tracking for students, and a drastic reduction in food wastage through data-driven cooking estimates.

1.2 Intended Audience and Document Overview

This document is intended for three primary audiences:

- **The Development Team (FullThrottle Tenacious Dynasty):** To understand the specific functional requirements, interface designs, and logic constraints required for implementation.
- **Mess Managers and Wardens (Users):** To visualize the proposed solution, understand how it integrates with their daily workflow, and validate that their operational needs are met.
- **Project Evaluators and Stakeholders:** To assess the feasibility, scope, and technical depth of the project.

Document Organization: The remainder of this Software Requirements Specification (SRS) is organized as follows:

- **Section 2 (Overall Description):** Provides a high-level overview of the product functionality, user characteristics, and system constraints.
- **Section 3 (Specific Requirements):** Details the functional requirements, external interfaces, and system logic (e.g., QR validation, rebate calculations) required for development.
- **Section 4 (Other Non-functional Requirements):** Outlines performance standards, security measures, and software quality attributes.

Recommended Reading Sequence:

- **Stakeholders and Users** are recommended to begin with the **Overall Description**, which provides a general understanding of the system's capabilities and user interactions, before reviewing the functional summary.

- **Developers and Testers** should proceed directly to the **Specific Requirements** and **Other Non-functional Requirements** sections to obtain the detailed technical specifications, API logic, and database schemas necessary for implementation.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
API	Application Programming Interface
CMMS	Centralized Mess Management System
CRUD	Create, Read, Update, Delete
Extras	Paid food items purchased separately from the fixed menu
Guest Token	Digital pass authorizing non-residents to dine
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IITK	Indian Institute of Technology, Kanpur
JWT	JSON Web Token
QR Code	Quick Response Code
Rebate	Official leave of absence from the mess with a refund
SRS	Software Requirements Specification
UI	User Interface

1.4 Document Conventions

This document adheres to the **IEEE 830 standard** for Software Requirements Specifications. The following conventions are applied throughout the document to ensure consistency and readability.

1.4.1 Formatting Conventions

- **Font:** Arial is used for all text throughout the document.
- **Font Sizes:**
  - **Body Text:** Size 11.
  - **Section Headings:** Size 18 (Bold).

    - **Subsection Headings:** Size 14 (Bold).
- **Spacing:** Single spacing is used for all text paragraphs.
- **Margins:** Standard 1-inch margins are maintained on all sides.

1.4.2 Typographical Conventions

- **Bold:** Used to highlight key system entities (e.g., **Student Portal**, **Manager Portal**), user interface elements (e.g., **Submit Button**), and important terms defined in the glossary.
- *Italics:* Used for comments, notes, or to emphasize specific states of data (e.g., *Order Pending*, *Delivered*).

1.4.3 Naming & Priority Conventions

The following priority levels classify functional requirements in Section 3:

- **High:** Critical features required for the core functioning of the system (e.g., Authentication, QR Generation).
- **Medium:** Important features that add significant value (e.g., Rebate Analytics).
- **Low:** Desirable features that can be implemented if time permits (e.g., Nutritional Tracker).

1.5 References and Acknowledgments

This software requirements specification relies on the following standards and guidelines:

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **CS253 Course Guidelines:** Course Project Documentation Guidelines (2025-26), Department of Computer Science and Engineering, IIT Kanpur.
- **IITK Hall Management Council Rules:** Operational guidelines regarding rebate policies, guest dining limits, and mess extras pricing.

Acknowledgments

We would like to acknowledge the guidance of our mentor, **Vinayak Bhosle**, for his support during the requirement analysis phase.

Overall Description

2.1 Product Overview

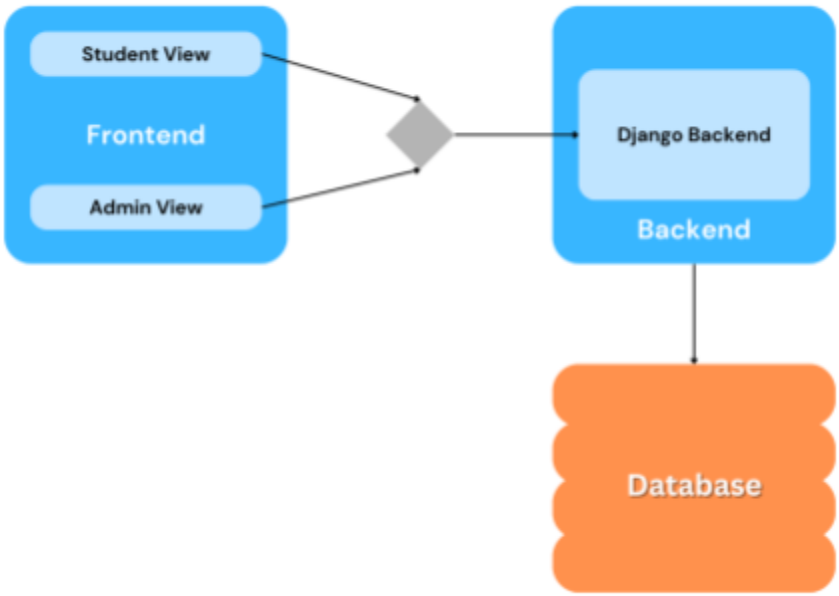
The **Centralized Mess Management System (CMMS)** is a web-based application designed to digitize and unify mess-related operations across multiple hostels/halls into a single centralized platform. Currently, mess operations such as menu display, extra-item booking, rebate (mess leave) application, and complaint handling are handled manually or through fragmented systems, leading to inefficiencies, long queues, data inconsistency, and food wastage. CMMS aims to replace these manual and semi-digital processes with a streamlined, transparent, and automated system.

CMMS is a **new, self-contained product** that serves as a centralized portal for students, mess managers, and mess staff. It acts as a bridge between students consuming mess services and administrators managing inventory, rebates, and feedback. The system provides role-based access, ensuring that students and managers interact only with the functionalities relevant to them. The software interfaces with standard web browsers on desktops and mobile devices and communicates with a backend server that handles authentication, business logic, and persistent data storage.

From a system perspective, CMMS consists of three primary components:

- A frontend web application used by students and managers
- A backend REST API that processes requests and enforces business rules
- A database layer that stores user data, menus, bookings, transactions, and analytics

These components interact over secure HTTP connections. The frontend sends requests to the backend API, which processes them and interacts with the database before returning responses.

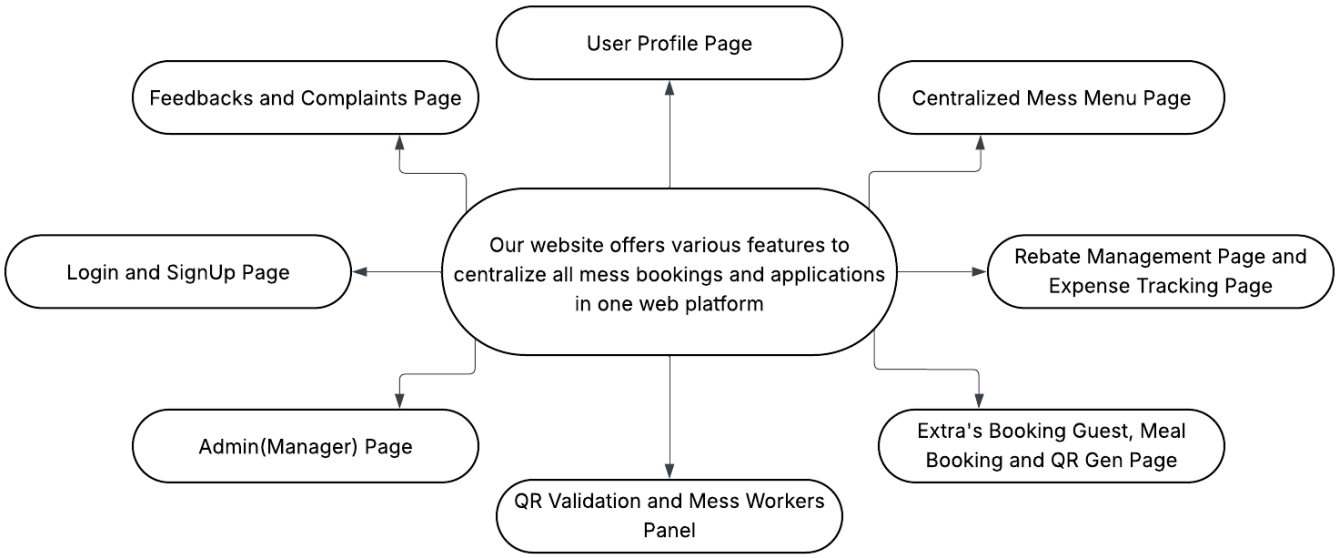


2.2 Product Functionality

The CMMS provides a comprehensive set of features aimed at automating mess operations and improving user experience for both students and mess administrators.

Major Functions of the System

- Centralized display of daily and weekly menus for all campus messes
- Online pre-booking of extra food items with QR code generation
- QR-based order verification and delivery confirmation
- Automated mess leave (rebate) application and refund calculation
- Guest meal booking with digital guest tokens
- Student-side expense tracking for extra items
- Complaint and feedback submission by students
- Complaint tracking and resolution management by mess administrators
- Live stock monitoring and automatic booking cutoff for special items
- Analytics dashboard for rebate forecasting and food wastage reduction
- Optional nutritional information display for fixed menus



2.3 Design and Implementation Constraints

The development and implementation of CMMS are subject to the following constraints:

- **Technology Stack Constraint**:
    - Frontend must be developed using **Vite + React.js**
    - Backend must be implemented using **Django with Django REST Framework**
    - Database must be **PostgreSQL** (preferred) or **SQLite** for development
    - Styling must follow **TailwindCSS** conventions
    - Animations should be implemented using **Framer Motion**
- **Hardware Constraints**:
    - The system must run efficiently on standard student and staff devices (mobile phones, tablets, and desktops)
    - QR code scanning should work with basic camera hardware available on smartphones
- **Security Constraints**:
    - Authentication must use **JWT-based secure login**
    - All communication must occur over **HTTPS**
    - Role-based access control must be enforced
- **Performance Constraints**:
    - The system must support concurrent booking requests without overselling inventory
    - Stock updates must reflect near real-time changes
- **Maintainability Constraints**:
    - Code must follow standard React and Django coding practices
    - Clear API separation must be maintained to allow future feature expansion
- **Operational Constraints**:
    - The system assumes continuous internet connectivity during usage
    - Offline functionality is not supported in the initial version

2.4 Assumptions and Dependencies

Assumptions

- All students and mess staff have access to smartphones or computers with internet connectivity
- Mess administrators are willing to adopt and consistently use the digital system
- Menu, pricing, and stock data are updated regularly by mess management
- QR code scanners (mobile cameras or dedicated devices) are available at mess counters
- Nutritional data (if implemented) is either manually entered or sourced from reliable references

Dependencies

- Dependence on third-party libraries such as:
    - `axios` for API communication
    - `qrcode.react` for QR code generation
    - `html5-qrcode` or `react-qr-reader` for QR scanning
    - `chart.js` for data visualization
- Dependence on:
    - Django REST Framework for API handling
    - PostgreSQL or SQLite for data persistence
    - Browser compatibility with modern JavaScript (ES6+)

Any change or failure in these assumptions or dependencies may impact system functionality, performance, or scope.

Specific Requirements

3.1 External Interface Requirements

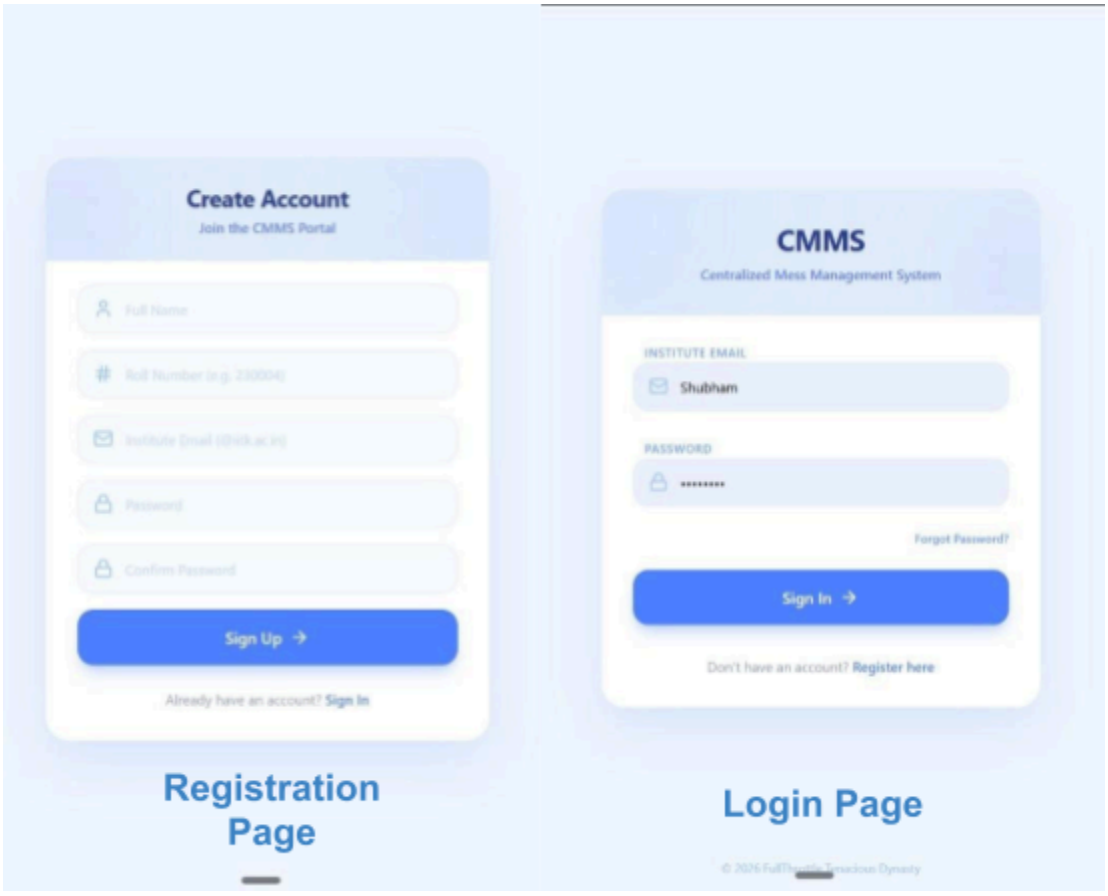
This section describes how the Centralized Mess Management System interacts with users, hardware, and other software systems.

3.1.1 User Interfaces

The Centralized Mess Management System shall provide a **web-based user interface** accessible through standard web browsers on desktops, laptops, tablets, and smartphones.

The user interface will be **role-based**, with different views and functionalities for **students, mess staff, and administrators**.

(a) Login and Registration Page



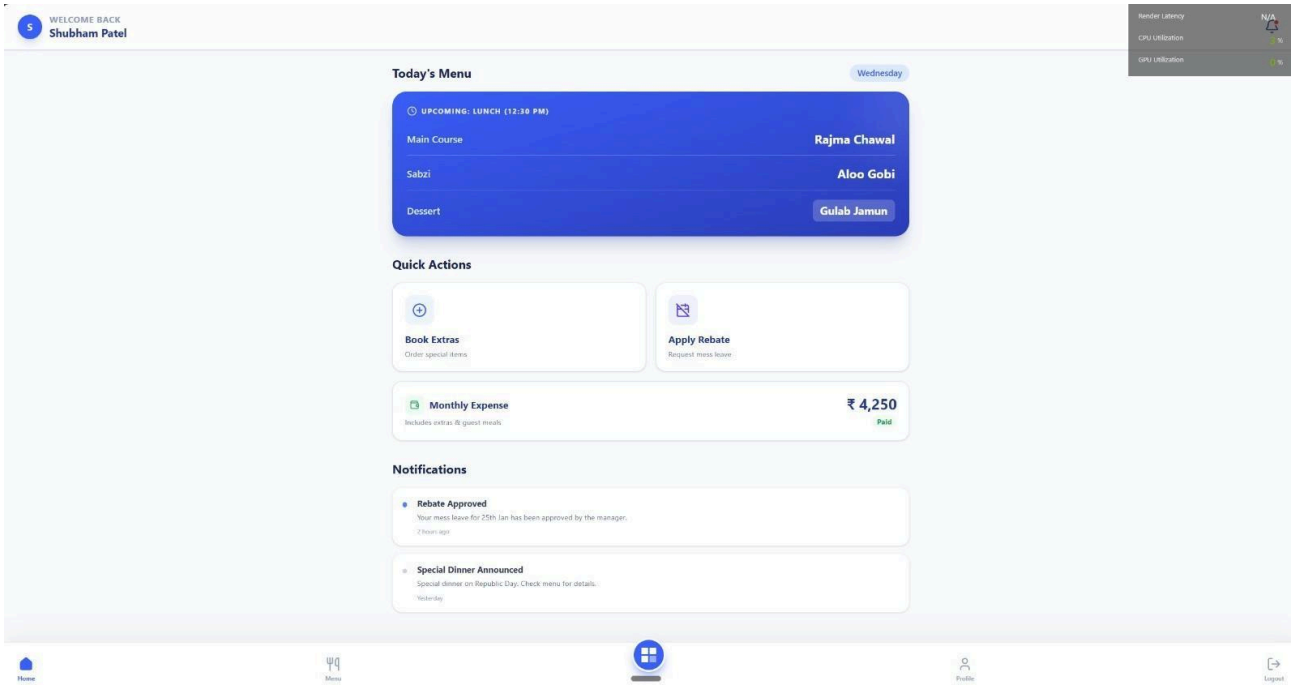
Logical Characteristics

- Provides secure login for students and mess administrators using institute email credentials.
- Contains input fields for Email ID and Password.
- Includes actions for Login, Sign Up, and Forgot Password.
- Supports role-based redirection after successful authentication.
- Displays validation messages for incorrect credentials or inactive accounts.

User Interaction

- Users enter their registered email ID and password.
- Clicking the Login button authenticates the user and redirects them to the appropriate dashboard.
- New users can navigate to the Sign Up page to create an account.
- Users who forget their password can request a reset via email.

(b) Student Home Dashboard



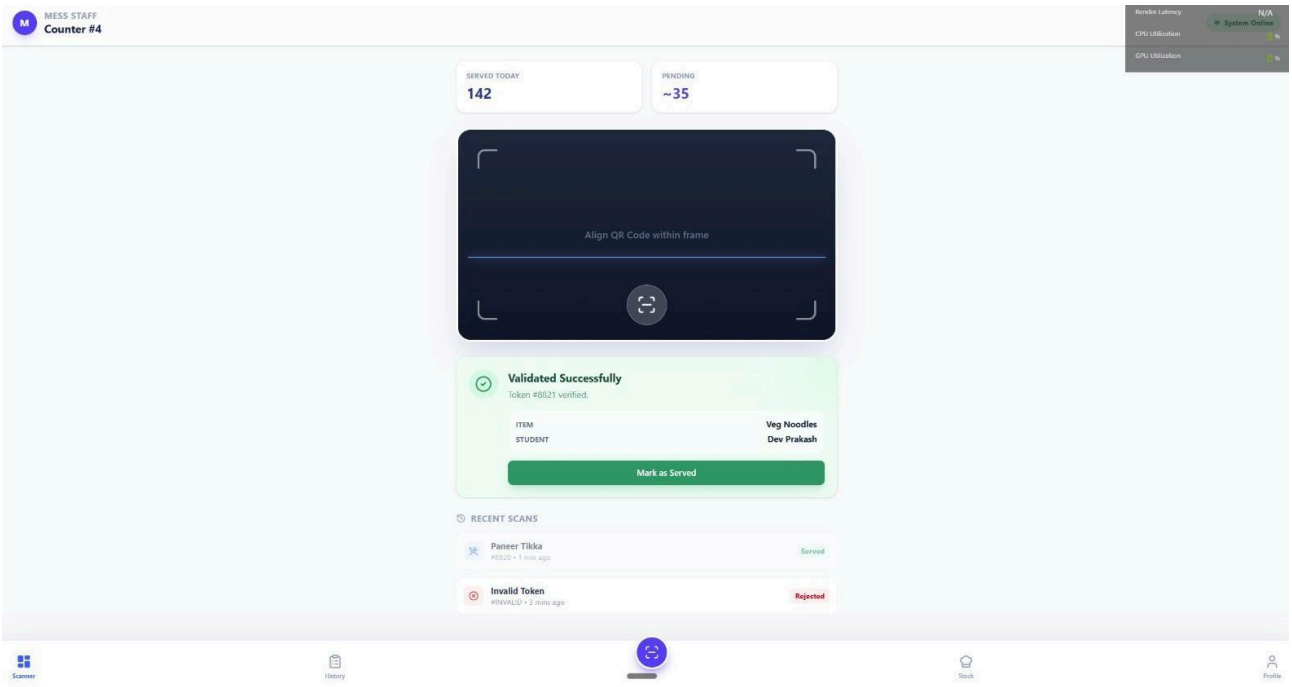
Logical Characteristics

- Displays an overview of the student's mess-related activities.
- Shows quick access cards for:
    - Today's Menu
    - Book Extra Items
    - Apply for Rebate
    - View Expenses
- Includes navigation bar for accessing profile, feedback, and logout options.
- Displays notifications related to bookings, rebates, or announcements.

User Interaction

- Students tap on any module card to navigate to detailed pages.
- Swipe or scroll to view menu items and announcements.
- Use bottom navigation or hamburger menu for switching sections.

(c) Mess Staff Dashboard



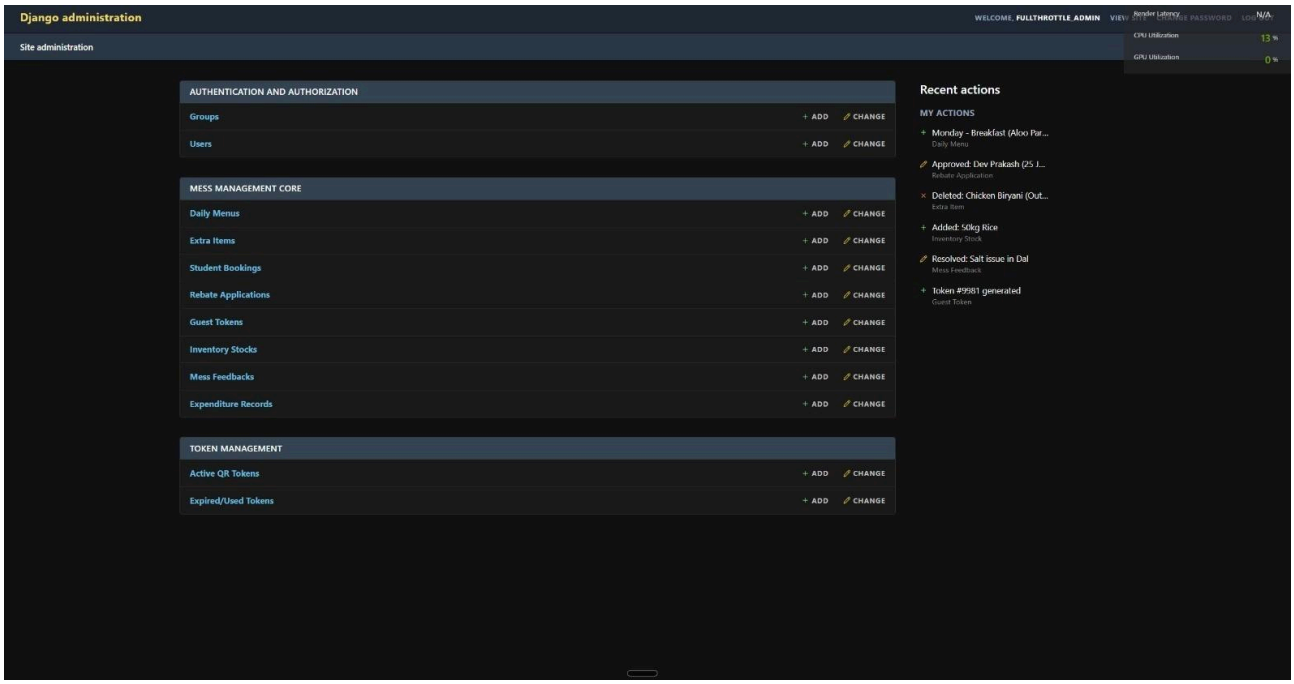
Logical Characteristics

- Provides a role-specific interface for mess staff to validate bookings.
- Displays a **QR code scanner** for extra items and guest meals.
- Shows booking details and updates order status to **Delivered** after successful scan.
- Automatically invalidates used or expired QR codes.
- Displays error messages for invalid or reused QR codes.

User Interaction

- Staff scan the student's QR code using the device camera.
- The system confirms validity or displays an error.
- Staff proceed to serve the item after confirmation.

(d) Manager Dashboard



Logical Characteristics

- Provides an administrative interface for managing mess operations.
- Displays expected meal counts, rebate summaries, and live stock levels.

- Allows menu updates and configuration of extra item availability.
- Shows analytics related to attendance, rebates, and food demand.
- Provides access to feedback and report generation.

User Interaction

- Managers navigate through dashboard sections using menus.
- Update menus and item limits using forms.
- View analytics and manage feedback statuses.

3.1.2 Hardware Interfaces

The Centralized Mess Management System does not require direct interaction with specialized hardware devices and primarily operates on **general-purpose computing hardware**.

The system shall be compatible with:

- Desktop and laptop computers
- Tablets and smartphones
- Servers or cloud infrastructure hosting the application

Optional hardware interactions may include:

- Barcode or QR-code scanners (for extra item verification)
- Printers (for printing reports or receipts)

All hardware interactions will be handled through standard operating system drivers and browser-supported interfaces. No custom hardware protocols are required.

3.1.3 Software Interfaces

This section defines the specific software connections, libraries, and communication protocols the Centralized Mess Management System (CMMS) utilizes to function.

- **Web Browser(Client Side Interface)**

- The client-side application is built using React.js and requires a modern web browser supporting ES6+ JavaScript standards to render the user interface and handle client-side logic.
- **Supported Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari
- **Interaction:** The browser executes the React application (built with Vite), handles routing via React Router (implied by Single Page Application structure), and manages local state.
- **Styling & Animation:** The browser renders styling processed from **TailwindCSS** and executes animations powered by **Framer-Motion.**

■

- **Web Server & Backend Interface**
  - The server-side logic interfaces with the Python runtime environment.
  - **Framework:** The system runs on **Django** , utilizing the **Django REST Framework (DRF)** for API endpoints.
  - **Runtime:** Requires a Python environment capable of handling concurrent requests and managing the Django application lifecycle.

- **Database Management System (DBMS)**
  The system interfaces with a robust database (PostgreSQL recommended for production) to ensure data integrity, handle concurrent transactions, and persist the application state. All database interactions are abstracted via the Django ORM to prevent SQL injection and ensure schema consistency.

  - **User & Authentication Data:** Stores secure credentials and manages session states using `loginStatus` tokens (stored as HTTP-only cookies). It enforces strict Role-Based Access Control (RBAC) via the `userRole` field to distinguish between Students, Developers, and Admins.

  - **Real-Time Inventory Management:** Manages the `stockCount` for "Extra" items with strict concurrency controls (ACID properties) to prevent negative inventory during simultaneous bookings by multiple students.

  - **Order & Transaction Lifecycle:** Tracks the complete lifecycle of a meal order via bookingStatus (Booked > Delivered > Cancelled) and stores unique, encrypted qrToken strings to validate claims and prevent theft.

  - **Financial & Rebate Records:** Maintains immutable records of student expenditures (`expenditureTotal`) and processes mess leave applications via `rebateStatus`, automatically validating submission deadlines (e.g., 24-hour notice).

  - **Menu & Feedback Synchronization:** Centralizes menu data across all hostels using messMenuID and tracks student grievances through a structured feedbackState workflow (Open >In Progress > Resolved).

- **External Library & Tool Integrations**

- The CMMS interfaces with specific third-party libraries to handle specialized tasks defined in the technical implementation strategy.
- **QR Code Generation (Client-Side)**: The Student Portal interfaces with the `qrcode.react` library to dynamically generate unique QR codes representing booked "Extra" items.
- **QR Code Scanning (Admin Interface)**: The Manager Portal interfaces with `html5-qrcode` or `react-qr-reader` to access the device camera and decode student QR codes for order validation and inventory deduction.
- **API Communication (Client-Server):** The frontend utilizes **`axios`** to handle asynchronous HTTP requests (GET, POST, PUT) to the Django backend. It manages request headers for authentication tokens (JWT) and intercepts responses to handle errors or token refreshing seamlessly.

- **Email & Notification Services**
  The system interfaces with Brevo (formerly Sendinblue) to handle the reliable delivery of transactional emails, ensuring critical communication reaches students and staff without being filtered as spam.
  - **Functionality:** Manages high-priority automated alerts including OTPs for secure login, Forget password queries, monthly "Mess Bill" PDF attachments, and real-time notifications for "Rebate Status" updates (Approved/Rejected).
  - **Integration:** Connects to the Django backend via **SMTP relay** or the **Brevo REST API** (using libraries like `django-anymail`). This allows the system to utilize Brevo's infrastructure for template management and delivery tracking (sent/opened/bounced).

All software interfaces will communicate using secure APIs and standard data formats such as JSON.

3.2 Functional Requirements

This section describes the detailed functional requirements of the Centralized Mess Management System. These requirements specify the system's expected behavior in terms of services, tasks, and functions provided to different users.

F1. User Authentication and Role Management

- The system shall allow users to register and log in using valid institute-issued credentials.
- The system shall authenticate users securely and maintain login sessions.
- The system shall enforce role-based access control for Students, Mess Staff, and Mess Administrators.
- The system shall redirect users to role-appropriate dashboards after login.

F2. User Profile Management

- The system shall allow users to view their profile information.
- The system shall allow users to update basic personal details where permitted.
- The system shall store all user information securely in the database.

F3. Menu Management

- The system shall display daily and weekly mess menus in a centralized manner.
- The system shall organize menus by day and meal type (breakfast, lunch, dinner).
- The system shall allow mess managers to update menus.
- The system shall reflect menu updates to students in real time.

F4. Extra Item Booking and QR Code Generation

- The system shall allow students to pre-book extra food items.
- The system shall verify item availability before confirming a booking.
- The system shall generate a unique, single-use QR code for each confirmed booking.
- The system shall update the booking status after confirmation.

F5. QR-Based Order Validation

- The system shall allow mess staff to scan QR codes presented by students.
- The system shall validate QR codes for authenticity, expiry, and usage status.
- The system shall mark bookings as *Delivered* after successful validation.
- The system shall automatically invalidate QR codes after use.

F6. Rebate (Mess Leave) Management

- The system shall allow students to apply for mess leave (rebate) for selected dates.
- The system shall validate rebate requests against submission deadlines.

- The system shall automatically calculate the applicable rebate amount.
- The system shall update the student's bill based on approved rebates.

F7. Guest Meal Booking

- The system shall allow students to book meals for guests.
- The system shall generate a digital guest token upon successful booking.
- The system shall enforce guest meal limits and booking deadlines.
- The system shall allow guest tokens to be validated during meal service.

F8. Student Expense Tracking

- The system shall track student expenditure on extra food items.
- The system shall update expenses immediately after successful bookings.
- The system shall allow students to view a read-only summary of monthly expenses.

F9. Feedback and Complaint Management

- The system shall allow students to submit feedback and complaints.
- The system shall store complaints with status tracking (Open, In Progress, Resolved).
- The system shall allow mess managers to review and update complaint statuses.
- The system shall notify students when complaint status changes.

F10. System Administration

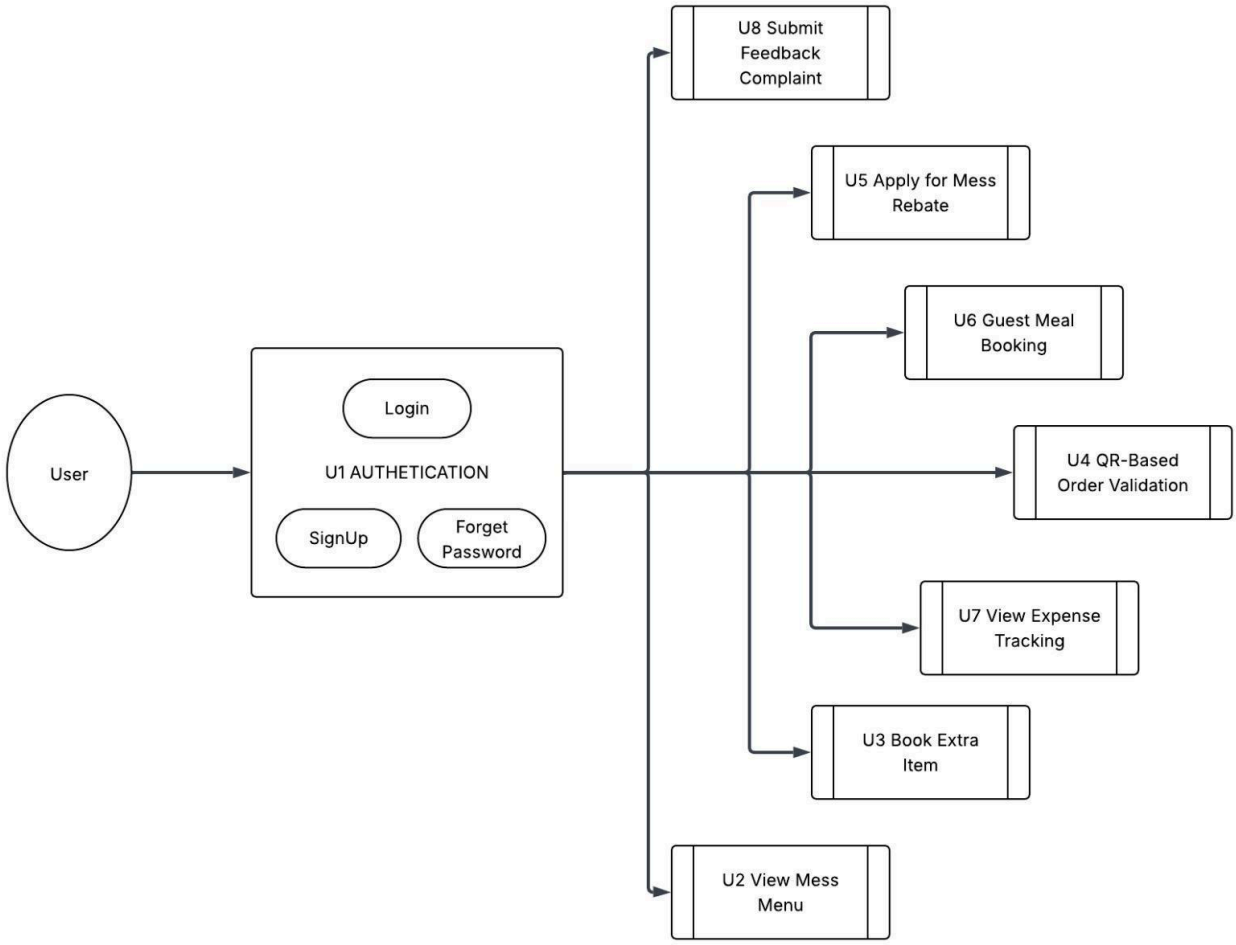
- The system shall allow administrators to manage user accounts, roles, and system configurations.
- The system shall maintain logs of important system activities.

3.3 Use Case Model

The Use Case Model illustrates the functional interactions between the **Centralized Mess Management System** and its actors. The system primarily interacts with **Students**, **Mess Staff**, and **Administrators**. Each actor performs specific actions such as viewing menus, managing attendance, handling billing, and generating reports.

The use case diagram encapsulates all major system functionalities including user authentication, menu management, and feedback handling. These use cases collectively represent the complete behavior of the system from a user's perspective.

Figure: Use Case Diagram



3.3.1 Use Case 1: User Login

Use Case ID	U1
Author	FullThrottle Tenacious Dynasty
Purpose	To allow users (students and mess administrators) to securely log

	into the system and access role-based functionalities.
Requirements Traceability	F1 (User Authentication), F2 (User Profile Management)
Priority	High
Preconditions	• User must be registered in the system • System must be online
Postconditions	• User is authenticated • User is redirected to the appropriate dashboard
Actors	Student, Mess Administrator, Server
Exceptions	• Invalid credentials • Account inactive • Network failure
Includes	Password validation, Role verification

3.3.2 Use Case 2: View Mess Menu

Use Case ID	U2
Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to view daily and weekly mess menus in a centralized manner.
Requirements Traceability	F3 (Menu Management)
Priority	High
Preconditions	User must be logged in
Postconditions	Menu is displayed to the user
Actors	Student, Server
Exceptions	Menu not updated
Includes	None

3.3.3 Use Case 3: Book Extra Items

Use Case ID	U3

Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to pre-book extra food items and receive a QR code for collection.
Requirements Traceability	F4 (Extras Booking & QR Generation), F8 (Student Expense Tracking)
Priority	High
Preconditions	• Student must be logged in • Extra item must be available
Postconditions	• Booking confirmed • QR code generated • Expense updated
Actors	Student, Server, Database
Exceptions	• Item sold out • Database failure
Includes	Inventory check, QR code generation

3.3.4 Use Case 4: QR-Code Based Order Validation

Use Case ID	U4
Author	FullThrottle Tenacious Dynasty
Purpose	To allow mess staff to validate student bookings using QR codes.
Requirements Traceability	F5 (QR-Based Order Validation)
Priority	High
Preconditions	• Valid QR code • Booking exists
Postconditions	• Order marked as delivered • QR invalidated
Actors	Mess Staff, Server
Exceptions	• Invalid or expired QR • Already used QR
Includes	Booking verification

3.3.5 Use Case 5: Apply for Mess Rebate

Use Case ID	U5
Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to apply for mess leave and receive automated rebates.
Requirements Traceability	F6 (Rebate Management)
Priority	Medium
Preconditions	• Student logged in • Request within deadline
Postconditions	• Rebate approved/rejected • Bill updated
Actors	Student, Server
Exceptions	• Deadline missed • Invalid date
Includes	Rebate calculation

3.3.6 Use Case 6: Guest Meal Booking

Use Case ID	U6
Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to book meals for their guests using a digital guest token generated by the system.
Requirements Traceability	F7 (Guest Meal Booking)
Priority	Medium
Preconditions	• Student must be logged in • Guest meal booking must be allowed for the selected date
Postconditions	• Guest meal booking is confirmed • A digital guest token is generated
Actors	Student, Server
Exceptions	• Guest booking limit exceeded • Booking deadline missed
Includes	Token generation, booking validation

3.3.7 Use Case 7: View Expense Tracking

Use Case ID	U7
Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to view and track their monthly expenditure on extra food items.
Requirements Traceability	F8 (Expense Tracking)
Priority	Medium
Preconditions	Student must be logged in
Postconditions	Expense summary is displayed to the student
Actors	Student, Server, Database
Exceptions	No expense data available
Includes	Expense calculation, data retrieval

3.3.8 Use Case 8: Submit Feedback/ Complaint

Use Case ID	U8
Author	FullThrottle Tenacious Dynasty
Purpose	To allow students to submit feedback or complaints related to mess services and allow administrators to manage and resolve them.
Requirements Traceability	F9 (Feedback and Complaint Management)
Priority	Medium
Preconditions	Student must be logged in
Postconditions	• Feedback/complaint is recorded in the system • Status is set to *Open*
Actors	Student, Mess Administrator, Server
Exceptions	Submission failure due to network issues
Includes	Complaint tracking, status updates

Other Non-functional Requirements

4.1 Performance Requirements

The Centralized Mess Management System (CMMS) shall satisfy the following performance requirements to ensure that digital operations do not introduce delays compared to the existing manual processes, especially during peak meal hours.

1. **Scalability & Concurrency:** The system shall support concurrent access by a large number of students and mess staff during peak meal times (breakfast, lunch, and dinner) without system failure or noticeable degradation in responsiveness.
2. **Response Time:** The Student Portal shall display daily and weekly menus across all halls within 2 seconds under normal campus network conditions.
3. **Response Time:** The confirmation of extras bookings and generation of the corresponding QR code shall be completed within 1 second after a successful booking.
4. **Response Time & Reliability:** QR code scanning and validation at the mess counter shall take no more than 1 second from scan to confirmation.
5. **Real-Time Data Updation:** Updates to the availability of limited extras items shall be reflected in the system in real time, ensuring that bookings are stopped immediately once stock reaches zero.
6. **Automation & Performance Efficiency:** Rebate (mess leave) applications shall be processed instantly by the system, with the calculated rebate reflected in the student's bill without manual approval delays.

Rationale:
These performance requirements are necessary to prevent queue formation at mess counters and to ensure that the digital system improves, rather than hinders, the existing dining workflow.

4.2 Safety and Security Requirements

The CMMS shall include safeguards to prevent unauthorised access, data misuse, and fraudulent dining claims, while protecting user privacy.

1. The system shall require all users to authenticate using **institute-issued email credentials** before accessing any functionality.
2. The system shall enforce **role-based access control**, ensuring that:
    a. Students can only access their own bookings, rebates, expenditure, and QR codes.
    b. Mess workers can only scan and validate QR codes and view relevant order details.
    c. Mess management and administrators can access analytics, stock data, and rebate summaries.
3. All QR codes generated for extras bookings and guest dining shall be:
    a. **Single-use**
    b. **Time-limited**
    c. Automatically invalidated after successful scanning or expiry
4. The system shall prevent the reuse or duplication of QR codes to avoid multiple claims for the same meal or extra item.

5. All communication between the client and server shall be conducted over **secure encrypted connections (HTTPS)**.
6. The system shall maintain **audit logs** for critical actions such as bookings, QR validations, and rebate processing to support accountability and dispute resolution.
7. Personal data related to students and guests shall be stored securely and shall not be accessible to unauthorised users.

4.3 Software Quality Attributes

4.3.1 Reliability

1. The system shall ensure that confirmed extras bookings, guest tokens, and rebate applications are **not lost** due to software or network failures.
2. The system shall remain operational during all scheduled meal hours.
3. Automated backups of booking, rebate, and expenditure data shall be maintained to allow recovery in case of system failure.

Achievement Strategy:
Reliability will be achieved through transactional database operations, server-side validation, and periodic automated backups.

4.3.2 Usability

1. The system shall provide a **simple and intuitive user interface** suitable for frequent daily use by students and mess staff.
2. Students shall be able to complete common actions (extras booking, rebate application) in **three or fewer steps**.
3. Mess workers shall be able to validate orders using QR scanning with **minimal interaction**.
4. The interface shall be fully responsive and usable on mobile devices, as students primarily access the system via smartphones.

Preference:
Ease of use is prioritised over advanced customisation, as the system is intended for repeated use by non-technical users.

4.3.3 Maintainability

1. The system shall be designed using a **modular frontend-backend architecture** to allow independent updates.
2. Changes to menus, extras, or pricing shall not require modifications to the core business logic.
3. The system shall support the addition of new halls, menu categories, or analytics features without significant redesign.

Achievement Strategy:
Maintainability will be ensured through modular UI components, well-defined APIs, and a structured database schema.

Other Requirements

5.1. Database Requirements

- **Preventing Double Bookings:** The system must guarantee that if two students try to book the last available item (e.g., the last Veg Noodles) at the exact same moment, only one of them will succeed. The other student should receive a "Sold Out" message immediately.
- **Accurate Inventory:** The "Live Stock Counter" must always reflect the real number of items available. It should never be possible for the number of bookings to exceed the amount of food actually prepared.
- **Transaction Safety:** If a booking fails (for example, due to internet issues or sold-out stock), no money should be deducted from the student's expenditure account.
- **Data Retention:** The system should retain student "Expenditure" and "Rebate" history for at least one full academic semester.
- **Backup:** Automated daily backups of the booking data should be performed to prevent loss of financial or meal records.

5.2. Hardware & Interface Requirements

- **Manager Interface Compatibility:** The **Order Validation** module relies on scanning QR codes. The Manager Portal must be compatible with standard mobile browsers (Chrome/Firefox) and have permission to access the device camera via `react-qr-reader`.
- **Display Responsiveness:** The Student Portal must be fully responsive and optimized for mobile devices, as students will primarily use phones to generate booking QR codes at the mess counter.

5.3. Legal and Privacy Requirements

- **Identity Protection:** Since the system uses "Guest Tokens" and student IDs, Personal Identities must be handled according to the institute's data privacy policies.
- **Financial Auditability:** As the system tracks "Automated Rebates" and "Expenditure", it acts as a financial ledger. All transactions must be immutable (no deleting past transaction records) to ensure the mess bill calculation is legally defensible.

Appendix A – Data Dictionary

Field Name	*Description*	*Possible States/Values*	*Operations/Functionality*	*Requirements*
userRole	Defines the level of access a user has while interacting with the software	Student, Mess Staff, Mess Admin	Determines access to features (e.g., Booking vs. Accessing the admin panel).	*Role-based restrictions must be strictly enforced.*
loginStatus	Tracks the login status of the user	Logged In/Logged Out(Boolean)	Determines whether the user is logged in or logged out.	*Tokens must be stored securely in the form of cookies.*
stockCount	Tracks the real-time availability of limited "Extra" items (e.g., Veg Noodles).	Must be greater than or equal to zero. (Cannot be negative)	Decrements on student booking; Stops bookings at 0.	*Must handle concurrent requests to prevent negative stock.*
bookingStatus	The current lifecycle state of a meal or extra item order.	Booked, Delivered, Cancelled	Updated via QR scan.	*Must not transition to "Booked" after "Delivered".*

rebateStatus	Tracks the status of a student's application for a rebate (mess leave).	Pending, Approved, Rejected	The system automatically validates date logic and updates the bill.	Must be applied at least one day before the rebate date.
qrToken	A unique, encrypted string used to validate a student's order at the counter.	UUID / Encrypted String	Generated upon booking; Scanned for validation.	*Must be unique and single-use to prevent theft.*
expenditureTotal	Accumulates the total cost of "Extras" purchased by a student for the month.	Decimal / Float (Currency)	Updates immediately upon booking confirmation.	*Must be immutable (cannot be manually edited).*
messMenuID	Identifies the specific menu items available for a specific week and hostel.	*Day (Mon-Sun), MealType (Breakfast/Lunch/Dinner)*	*Retrieved by the Student Portal to display options.*	*Must sync with the central database for all hostels.*
feedbackState	Tracks the progress of a complaint lodged by a student regarding food/service.	Open, In Progress, Resolved	Manager/Admin updates the state after taking action.	*Visible to the student who lodged it and the admin.*

Guest QrToken	A unique digital pass generated for a non-resident guest to authorize dining.	UUID / Encrypted String	Generated upon successful booking; Scanned by mess staff for validation.	*Must be single-use and strictly tied to a specific date and meal type.*
feedbackDescription	The detailed text content of a complaint or suggestion submitted by a student.	String / Text	Input by the student; Read-only for Mess Managers.	*Should support a character limit (e.g., 500 chars) to ensure conciseness.*
feedbackCategory	Classifies the nature of the complaint for easier sorting and reporting.	Food Quality, Hygiene, Staff Behavior, Infrastructure	Selected by the student from a dropdown list during submission.	*meaningful analytics on mess performance.*
submissionDate	The timestamp recording when a feedback or complaint was lodged.	DateTime	Automatically generated by the system upon submission.	*Used to track resolution time against service level expectations.*

Appendix B - Group Log

Date & Time	Venue	Activity
09/01/26 **7:00 PM**	Online Meet	Discussion related to the project ideas. Proposal of various project ideas by the team members.
10/01/26 **8:00 PM**	4th floor Rajeev Motwani Building	Met the Professor to discuss the various proposed ideas, finally deciding to continue with CMMS.
11/01/26 **9:30 PM**	Online Meet	Discussed the project idea and decided the software tools and frameworks for its implementation.
18/01/26 **5:00 PM**	1st floor Rajeev Motwani Building	Discussion related to the division of work for the SRS.
24/01/26 **9:30 PM**	Online Meet	Final review of the prepared SRS before submission.